



Disciplina Engenharia de Software

PROJETO – Parte Final

Este trabalho final da disciplina de Engenharia de Software tem como objetivo principal o desenvolvimento de um Sistema de Biblioteca, aplicando as melhores práticas de desenvolvimento ágil e integração contínua/entrega contínua (CI/CD) por meio de conceitos de DevOps. O projeto visa não apenas a implementação das funcionalidades do sistema, mas também a demonstração de um fluxo de trabalho colaborativo e automatizado.

Regras Gerais e Avaliação

- O trabalho pode ser realizado em equipes de até 4 pessoas.
- Cada membro será avaliado individualmente.
- Para cada dia de atraso, será decrescido 15 (quinze) pontos na nota final do trabalho.
- Para trabalhos que contenham artefatos iguais, será atribuído nota zero no artefato de todos os grupos envolvidos.
- O código será avaliado com a ferramenta RefactoringMiner, e caso haja códigos iguais, será atribuída a nota zero em todos.

Requisitos do Projeto: Sistema de Biblioteca

1. Arquitetura Proposta

Desenvolver e documentar a arquitetura do Sistema de Biblioteca, contemplando:

- a) **Visão Conceitual da Arquitetura:** Uma representação de alto nível da estrutura do sistema.
- b) **Descrição dos Elementos da Arquitetura e Dependências:** Detalhamento dos componentes, módulos e suas interações.
- c) **Padrões Arquiteturais:** Descrever qual ou quais padrões arquiteturais foram a base para a escolha da arquitetura (Exemplo: SOA, MVC, Camadas, Microserviços) e justificar a escolha.



2. Funcionalidades Essenciais

- a. **Cadastro de Livros e Alunos:** Implementar funcionalidades completas para cadastrar livros e alunos, incluindo uma interface gráfica (GUI) intuitiva para interação do usuário.
- b. **Finalizar a implementação do Caso de Uso Emprestar Livro:**
 - Para tanto, utilize o código disponível em <https://github.com/menolli/EngenhariaDeSoftware/tree/main/Codigo2> e siga a descrição dos exercícios 2, 3, 4, 5 e 6 da página 197.
 - Atualização do diagrama de classe, inserindo a camada de persistência aplicando o padrão DAO.
 - Finalização do Diagrama de Sequência apresentado na Figura 11.11 (página 226) para que tenha todas as funcionalidades para implementar o Caso de Uso.
 - **Implementação:** Desenvolver o caso de uso de acordo com as regras definidas e os novos diagramas de classes e sequência. Para isso, crie uma base de dados em um SGBD de sua escolha para ser o método de armazenamento persistente.
- c. **Fazer a implementação do Caso de Uso Devolver Livro, e essa deve contemplar:**
 - Descrever o caso de uso para esta funcionalidade
 - Atualização do diagrama de classe, inserido a camada de persistência aplicando o padrão DAO.
 - Fazer o diagrama de sequência da funcionalidade.
 - Implementar o caso de uso, de acordo com as regras definidas no caso de uso, e com os novos diagramas de o diagrama de classes e sequência (para isso crie uma base de dados em um SGBD da sua escolha para ser o método de armazenamento persistente).

4. Testes

- **Casos de Testes:** Implementar casos de testes unitários para as funcionalidades de emprestar e devolver.
- Criar testes automatizados para os casos de teste criados.

5. Persistência de Dados

- **Padrão DAO:** A camada de persistência deve obrigatoriamente utilizar o padrão Data Access Object (DAO). A escolha da tecnologia para a persistência é livre.



Requisitos de DevOps e Desenvolvimento Ágil

O projeto deve seguir um fluxo de trabalho que incorpore práticas de Desenvolvimento Ágil (Scrum/Kanban) e DevOps, utilizando o GitHub como plataforma central.

1. Configuração Inicial (DevOps Básico)

- **Repositório GitHub:** Criar um repositório público no GitHub para o projeto.
- **Membros da Equipe:** Adicionar todos os membros da equipe ao repositório.
- **README:** Definir um arquivo README.md claro com a descrição do projeto, instruções de setup e execução.

2. Planejamento com Scrum

- **Product Backlog:** O *produto backlog* está definido na seção anterior. As funcionalidades, são cadastrar livro e aluno, emprestar livro e devolver livros
- **Sprint:** Definir e executar as sprints de acordo com as funcionalidades exigidas para o sistema.

3. Uso de Kanban com GitHub Projects

- **Quadro Kanban:** Criar um quadro Kanban no GitHub Projects com as seguintes colunas mínimas: To Do, In Progress, Review, Done.
- **Gestão de Tarefas:** Cada tarefa deve estar representada no quadro e ter seu status atualizado conforme a evolução. As tarefas devem ser associadas aos responsáveis.

4. Uso de GitHub Issues

- **Criação de Issues:** Criar uma Issue para cada funcionalidade ou tarefa a ser desenvolvida.
- **Labels e Responsáveis:** Utilizar labels (ex: feature, bug, test, review) e associar responsáveis a cada Issue.
- **Comunicação:** Utilizar os comentários das Issues para comunicação e registro de decisões da equipe.
- **Vinculação:** As Issues devem estar vinculadas ao Project (Kanban).



5. Desenvolvimento do Sistema

- **Implementação:** Implementar as funcionalidades do Sistema de Biblioteca conforme os requisitos acima.
- **Branches:** Utilizar branches para o desenvolvimento de novas funcionalidades (ex: feature/cadastro-livro, feature/emprestar-livro).
- **Commits:** Realizar commits frequentes e descritivos, seguindo boas práticas de mensagens de commit.

6. Teste e Revisão

- **Tarefas de Teste:** Criar e executar as tarefas de teste para validar as funcionalidades conforme descrito na seção anterior.
- **Revisão de Código:** Realizar, no mínimo, 2 tarefas de revisão de código (Code Review) entre os membros da dupla.
- **Pull Requests:** Realizar a revisão de código via Pull Requests (PRs) no GitHub.

7. Integração com GitHub Actions (DevOps)

- **Automação:** Configurar pelo menos uma automação com GitHub Actions. Esta automação deve ser executada automaticamente quando um Pull Request for aberto. Exemplos de automação incluem:
 - * Executar build ou testes automaticamente.
 - * Enviar notificação (ex: log de execução).
 - * Validar Pull Requests (ex: verificar se o código compila, rodar testes unitários).
- b. **Pipeline CI:** O objetivo é criar um pipeline simples de Integração Contínua (CI).

8. Release

- **Criação de Release:** Criar uma versão do sistema (Release) no GitHub ao final do projeto.
- **Descrição da Release:** Descrever claramente o que foi entregue nesta versão.
- **Conclusão de Tarefas:** Garantir que todas as tarefas relacionadas à Release estejam finalizadas no GitHub Project.



Entregáveis

Os seguintes artefatos devem ser entregues:

- Link do repositório GitHub (contendo todo o código-fonte, histórico de commits, branches).
- Projeto (Kanban) configurado e utilizado no GitHub Projects.
- Issues organizadas e vinculadas ao Kanban.
- Código funcional do Sistema de Biblioteca, implementando todos os requisitos.
- Workflow do GitHub Actions configurado e em funcionamento.
- Release publicada no GitHub.
- Documentação da arquitetura (incluindo visão conceitual, elementos, dependências e padrões arquiteturais).
- Diagramas de Classe e Sequência atualizados e finalizados.
- Casos de teste implementados.

Tabela com os Critérios de Avaliação		
Valor Máximo	Critérios	
0,5	Descrição da Arquitetura do Sistema	
0,5	Funcionalidades cadastrar Livros e Alunos	
2,5	Funcionalidade Emprestar Livro	
0,5		<i>Atualização do digrama de classe</i>
0,5		<i>Diagrama de Sequência</i>
1,5		<i>Implementação do caso de uso</i>
2,5	Funcionalidade Devolver Livro	
0,5		<i>Diagrama de Caso de Uso</i>
0,5		<i>Diagrama de Classes</i>
0,5		<i>Diagrama de Sequência</i>
1,0		<i>Implementação do caso de uso</i>
1,0	Uso do Padrão DAO e Interface Gráfica do Sistema	
1,0	Testes	
0,5		<i>Casos de Testes</i>
0,5		<i>Testes Unitários automatizados</i>
2,0	DevOps	
0,5		<i>Planejamento dos Sprints com Kanban e Git Projects</i>
0,5		<i>Uso correto do GitHub Issues</i>
0,5		<i>Uso correto de Branches, commits e pull requests</i>
0,5		<i>Automação e Integração com o Gt Actions</i>